Python Day 2

자료구조와 함수

HYUNDAI AI Insight Campus · 온디바이스 AI · 프로그래밍 언어(Python) 2/4

2026. 9. 3 (목) 09:00-18:00 · 8교시 · Google Colab

리스트·튜플 | 딕셔너리·집합 | 함수 | 람다·컴프리헨션 | 모듈 | 예외 처리 | 시뮬레이터

강사: 이몽룡 · 길동

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1

리스트·튜플

참조 공유와 복사(얕은/깊은)를 구분하고 정렬·언패킹을 쓴다

2

딕셔너리·집합

JSON 형태 센서 데이터를 저장·조회·집계한다

3

함수

인자·반환값·스코프를 올바르게 쓰고 가변 기본값 함정을 피한다

4

람다·컴프리헨션

sorted(key=...)와 한 줄 변환으로 데이터를 다듬는다

5

모듈·예외

코드를 sensors.py 로 분리하고 try/except 로 깨진 데이터를 막는다

6

종합

센서 시뮬레이터 + 통계 리포트를 완성한다

시간표

8교시 × 50분 · 점심 12:00–13:00 · 모든 실습은 Google Colab

교시	시간	주제	형태	애니메이션
1	09:00–09:50	리스트와 튜플 — 참조와 복사	강의 + 실습	02 참조 모델 [Day 2], D2-3 리스트 메서드, 05 슬라이스 대입
2	10:00–10:50	딕셔너리와 집합	강의 + 실습	D2-1 해시 조회, D2-2 벤 다이어그램, D2-5 트리 뷰어
3	11:00–11:50	함수 기초	강의 + 실습	03 [Day 2] 함수 호출과 스택
4	13:00–13:50	람다·컴프리헨션	강의 + 실습	D2-4 컴프리헨션 변환기
5	14:00–14:50	모듈·표준 라이브러리	강의 + 실습	(라이브 시연 %%writefile)
6	15:00–15:50	예외 처리	강의 + 실습	04 실패 프리셋, 03 [Day 2] 예외 전파
7	16:00–16:50	종합 실습 — 센서 시뮬레이터	실습	D2-5 트리 뷰어, D2-6 결정 트리
8	17:00–17:50	정리·퀴즈·제출	평가	해설용 전체

오늘 사용하는 학습 도구 3종

예측 → 애니메이션에서 확인 → Colab에서 코드로 검증 → 자동 채점 — Day 1과 동일

애니메이션 10개

…/anim/index.html

- 신규 6개(day2) + Day 1 확장 4개
- 프리셋 버튼으로 시나리오 재생
- Day 1 파일에는 [Day 2] 프리셋이 추가됨
- 학생도 같은 주소로 접속 — 실습 중 직접 조작

Colab 워크시트

Day2_worksheet.ipynb

- 교시별 따라하기 → TODO → ✅ 자동 채점 → 🔍 자기 점검
- TODO는 raise NotImplementedError 부터
- 과제 A는 학생 데이터 기준 정합성 채점
- 마지막 '제출 요약' 셀 출력을 남기고 제출

개념 워크시트

…/anim/worksheet.html

- 교시별 10문항 × 7 = 70문항 (Day 2 전용)
- 채점 후 정오·해설·확인할 애니메이션 표시
- 제출 → 오답 정리 대화상자 → 캡처로 노션에
- 결과 코드를 Colab 제출 요약 셀에 입력

예측 → 애니메이션 확인 → 코드로 검증 → 자동 채점

애니메이션 — 신규 6개 + Day 1 확장 4개

다크 테마 공통 디자인 · 프리셋 버튼 → 캔버스 → 상태 패널 → 설명

D2-3 리스트 메서드 비교 — 1교시
append/extend/insert · sort() None 함정 vs sorted()

D2-1 딕셔너리 해시 조회 — 2교시
key → hash → 버킷 직행, 리스트 순차 탐색과 비교

D2-2 집합 벤 다이어그램 — 2교시
| & - ^ 영역 강조, 두 노드 감지 ID 비교

D2-5 중첩 자료구조 트리 뷰어 — 2·7교시
대괄호 하나 = 한 층 하강, 3종 오류 재현

D2-4 컴프리헨션 변환기 — 4교시
for 문 ↔ 한 줄 동시 하이라이트, 원소의 여행

D2-6 자료구조 결정 트리 — 2·7교시
4가지 질문 → list/tuple/dict/set 도착

02 변수·참조 모델 (확장) — 1교시
[Day 2] 중첩 얕은 복사 · deepcopy 프리셋 추가

03 제어 흐름 추적기 (확장) — 3·6교시
[Day 2] 함수 호출과 스택 · 예외 전파와 try

05 인덱싱·슬라이싱 (확장) — 1교시
[Day 2] 슬라이스 대입 a[1:3] = [9,9,9]

04 문자열 파싱 파이프라인 — 6교시
필드 누락·잘못된 숫자 — try/except 도입 동기

제목 클릭 시 열림 · Day 2 목차: https://devops0906.github.io/AI_CAMPUS/python-day2/anim/index.html (Day 1 확장 4개는 day1 폴더 — 갱신본 업로드 필요)

워크시트 사용법

Day 1과 동일 — Colab 노트북 하나로 실습·채점·제출

Colab 워크시트 — 교시당 한 섹션

1	⚙️ 설정	이름 입력 후 가장 먼저 실행 (NODES 데이터 포함)
2	따라하기	실행만 하고 출력을 읽는다
3	TODO 1~4	raise NotImplementedError 줄을 지우고 작성
4	✅ 자동 채점	몇 번이든 재실행 — PASS n/4 를 남긴다
5	🔍 자기 점검	드롭다운으로 예측 → 애니메이션에서 확인
6	막힌 점 한 줄	텍스트 셀에 기록

개념 워크시트 (HTML · Day 2)

- 교시별 10문항 · 채점하기 전까지 답 수정 가능
- 채점 후 정오 + 해설 + 확인할 애니메이션 프리셋
- 제출 → 오답 정리 대화상자 → 캡처로 노션에 붙여넣기
- 결과 코드 7개를 Colab '제출 요약' 셀에 입력

https://devops0906.github.io/AI_CAMPUS/python-day2/anim/worksheet.html

제출

```
이름: 홍길동
1교시      자동 채점 4/4
...
7교시-A    자동 채점 4/4
합계 32/32
개념 워크시트 코드: 1B-8052,2C-7dd4,...
```

온라인 자료 링크

브라우저에서 바로 열립니다 — 설치·다운로드 없음

기본 주소 (Day 2)

https://devops0906.github.io/AI_CAMPUS/python-day2/anim/

index.html	Day 2 애니메이션 목차 (신규 6 + 확장 4 링크)
worksheet.html	Day 2 개념 워크시트 (70문항 · 제출 · 캡처)

신규 애니메이션 (day2 폴더)

1교시 D2-3_리스트_메서드_비교.html

2교시 D2-1_딕셔너리_해시조회.html

2교시 D2-2_집합_벤다이어그램.html

2·7교시 D2-5_중첩자료구조_트리뷰어.html

2·7교시 D2-6_자료구조_결정트리.html

4교시 D2-4_컴프리헨션_변환기.html

Day 1 확장 4개 (day1 폴더): 02 참조 모델 · 03 추적기 · 05 슬라이싱 · 04 파이프라인

1 리스트와 튜플 — 참조와 복사

09:00–09:50

학습 목표

- 리스트 메서드와 sort() vs sorted() 의 차이
- b = a 참조 공유 / 복사 3형제 / 얕은 복사의 한계
- 튜플 불변성과 언패킹 (스왑, 별표)
- 슬라이스 대입 — 리스트에서만

🎬 **02 · D2-3 · 05 — 1교시 3종**

- 02: 리스트 참조 공유 → 복사 → [Day 2] 중첩 얕은 복사 → deepcopy
- D2-3: append vs extend → ⚠ a = a.sort() 함정 → sorted
- 05: [Day 2] 슬라이스 대입 a[1:3] = [9,9,9]
- 보라 화살표 = 객체 → 객체 참조 (중첩)

핵심 개념

참조 vs 복사

```
b = a           # 이름표 하나 더 (같은 객체)
c = a.copy()    # 새 리스트 (a[:], list(a) 동일)
copy.deepcopy(a)  # 안쪽까지 전부 복제
```

sort vs sorted

```
nums.sort()     → None, 원본 정렬
sorted(nums)    → 새 리스트, 원본 유지
a = a.sort()    # ⚠ a 가 None 이 되는 사고
```

튜플·언패킹

```
name, pin = ("LED", 3)    ·    a, b = b, a
mn, *rest = sorted(vals)
```

함정

```
[[0]*3]*2 — 두 행이 같은 안쪽 리스트를 공유
```

1 리스트와 튜플 — 실습

Colab 워크시트 TODO

1 TODO 1-1 정렬 두 방식
DAY1_TEMPS 상위 3개 top3 (원본 유지) → sort(reverse=True)

2 TODO 1-2 [[0]*3]*2 함정
wrong 과 right 를 만들어 [0][0]=1 비교, 이유를 텍스트 셀에

3 TODO 1-3 언패킹
mn, *rest = sorted(vals) · x, y 스왑

4 TODO 1-4 튜플 안 리스트 (심화)
record[1].append(27.0) — 왜 되는지 설명

✅ **자동 채점** top3/원본 · wrong/right · mn/rest/스왑 · record (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- b = a; b.append(3) 후 a → [1, 2, 3]
- a = a.sort() 후 a → None

체크포인트 · 강사 확인

- '복사'와 '이름표'를 구분해 말하는가
- [[0]*3]*2 를 참조 모델 그림으로 설명하는가
- sorted 와 sort 의 반환값을 즉답하는가

2 딕셔너리와 집합

10:00–10:50

학습 목표

- 딕셔너리 추가·덮어쓰기·삭제·순회
- KeyError vs .get(key, 기본값) vs in
- 중첩 구조 접근: dict → list → dict
- 집합 연산 | & - ^ 과 중복 제거

🎬 **D2-1 · D2-2 · D2-5 — 2교시 3종**

- D2-1: 조회 성공 → 없는 키 ✕ → .get(-1) → 리스트 키 ✕
- D2-1: 리스트 순차 탐색과 속도 비교 (O(1) vs O(n))
- D2-5: ["readings"][0]["T"] 한 층씩 → ✕ 층 착각
- D2-2: 교집합 & → 차집합 - → 중복 제거

핵심 개념

기본 조작

```
pins["BUZZ"] = 6    # 추가
pins["LED"] = 4     # 덮어쓰기 (키 중복 불가)
for name, pin in pins.items(): ...
```

없는 키 방어

```
pins["SERVO"]         # KeyError 로 중단
pins.get("SERVO", -1)  # 기본값 반환
"SERVO" in pins        # 존재 검사
```

중첩 접근

```
NODES["ESP32-01"]["readings"][0]["T"]
# dict 키 → dict 키 → list 인덱스 → dict 키
```

집합

```
a & b 공통 · a - b 한쪽만 · len(set(x)) 중복 제거
```

2 딕셔너리와 집합 — 실습

Colab 워크시트 TODO

1 TODO 2-1 중첩 접근·평균
ESP32-01 readings 의 T 평균 avg01 (for 문)

2 TODO 2-2 안전한 조회
pin = .get("SERVO", -1) · has = in 검사

3 TODO 2-3 집합 연산
감지 ID 공통 common(&) · A만 only_a(-)

4 TODO 2-4 카운트 패턴 (심화)
counts[level] = counts.get(level, 0) + 1

✅ **자동 채점** avg01 · pin/has · common/only_a · counts (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- d["없는키"] → KeyError
- 리스트가 키가 될 수 없는 이유 → 해시 불가

체크포인트 · 강사 확인

- '몇 층에 있고 그 층이 무슨 자료형인지'를 말하며 접근식을 쓰는가
- KeyError 사고를 .get() 으로 스스로 고치는가
- D2-6 결정 트리로 자료구조를 골라 보게 한다

3 함수 기초

11:00–11:50

학습 목표

- def · 호출 · return (없으면 None)
- 기본값·키워드 인자, *args / **kwargs
- 스코프 — 함수 안 대입은 지역 변수
- 가변 기본값 함정 def f(x, acc=[])

🎬 **03 제어 흐름 추적기 (확장)**

- [Day 2] 함수 호출과 스택 — judge ▸ temp 프레임 생성
- 변수 표에서 지역 변수가 생겼다 사라지는 순간
- return 과 함께 프레임 소멸 → status 에 대입
- 호출 전후 변수 표를 비교하게 한다

핵심 개념

정의와 호출

```
def adc_to_volt(raw, vref=3.3, bits=10):
    return raw / (2**bits - 1) * vref
adc_to_volt(512, vref=5.0)   # 키워드 인자
```

다중 반환

```
def stats(*values):
    return min(values), max(values), avg
lo, hi, avg = stats(22.1, 23.4, 31.2)
```

스코프

```
함수 안 대입 → 새 지역 변수 (전역 못 바꿈)
바꾸려면 반환값으로 주고받기 (global 지양)
```

가변 기본값

```
def f(v, acc=[])  ⚠ 호출 간 공유·누적 → acc=None 패턴
```

3 함수 기초 — 실습

Colab 워크시트 TODO

1 TODO 3-1 judge(temp)
Day 1 온도 규칙을 함수로 리팩토링

2 TODO 3-2 parse_field
"T=25.3" → ("T", 25.3) — 6교시에서 재사용

3 TODO 3-3 stats(*values)
(min, max, round(avg, 2)) 반환 + 언패킹

4 TODO 3-4 가변 기본값 (심화)
bad_log 함정 재현 → safe_log(acc=None) 로 수정

✅ **자동 채점** judge 5경계 · parse_field · stats · safe_log (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- return 없는 함수 → None
- *args 의 자료형 → 튜플

체크포인트 · 강사 확인

- print 와 return 을 구분하는가
- 경계값(0, 30)을 judge 에서 확인했는가
- 스택 프레임 그림으로 지역 변수 수명을 설명하는가

4 람다·컴프리헨션

13:00–13:50

학습 목표

- 람다 = key 자리에 넣는 한 줄 함수
- sorted(key=..., reverse=True) · max(key=...)
- [식 for 변수 in 시퀀스 if 조건]
- dict 컴프리헨션 · map/filter 는 이터레이터

🎬 **D2-4 컴프리헨션 변환기**

- 기본 — 2배 → 조건 — 짝수만 (탈락 원소는 식 계산 안 함)
- ADC → 전압 — Day 1 공식을 리스트 전체에
- for 문과 한 줄이 같은 순간에 하이라이트
- 문자열 파싱 — 짧은 것보다 읽히는 것이 우선

핵심 개념

key 정렬

```
sorted(records, key=lambda r: r["T"], reverse=True)
max(records, key=lambda r: r["T"])["id"]
```

for ↔ 한 줄

```
volts = []                          # for 문 3줄
for r in raws:
    volts.append(round(r/1023*3.3, 2))
volts = [round(r/1023*3.3, 2) for r in raws]
```

조건 필터

```
[t for t in temps if t > 35]
{r["id"]: r["T"] for r in records if r["T"] > 30}
```

읽는 순서

```
for → if → 맨 앞의 식  · 결과는 항상 새 리스트
```

4 람다·컴프리헨션 — 실습

Colab 워크시트 TODO

1 TODO 4-1 ADC → 전압
raws → volts 한 줄 [round(r/1023*3.3, 2) ...]

2 TODO 4-2 정렬·최고
by_T 내림차순 · hottest = max(key=...)["id"]

3 TODO 4-3 필터
T > 30 인 레코드만 hot 에

4 TODO 4-4 dict 컴프리헨션 (심화)
{노드id: T 평균} → id2avg

✅ **자동 채점** volts · by_T/hottest · hot · id2avg (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- [x*2 for x in [1,2,3] if x>1] → [4, 6]
- max(key=...) 반환 → 레코드 자체

체크포인트 · 강사 확인

- 컴프리헨션을 for 문으로 되돌려 말할 수 있는가
- 2중 컴프리헨션 남용을 자제시킨다 (2중까지만)
- map 결과를 print 해 이터레이터임을 확인시킨다

5 모듈·표준 라이브러리

14:00–14:50

학습 목표

- %%writefile 로 모듈 생성 → import
- 모듈 캐시와 importlib.reload
- if __name__ == "__main__": 자체 테스트
- Counter · datetime — 표준 라이브러리 활용

🎬 **라이브 시연 (Colab)**

- %%writefile → !python → import 순서로 직접
- __main__ 블록이 import 때 안 도는 것 확인
- 고친 뒤 reload 없이 import → 그대로임을 보여주기
- Counter 로 판정 집계 한 줄

핵심 개념

모듈 만들기

```
%%writefile sensors.py
def judge(temp): ...
def parse_field(field): ...
```

두 실행 방식

```
!python sensors.py      # __main__ 블록 실행
import sensors           # 실행 안 됨
sensors.judge(25.3)
```

캐시 주의

```
수정 후 import 재실행 → 반영 안 됨
importlib.reload(sensors) 또는 런타임 재시작
```

표준 라이브러리

```
Counter(statuses) · datetime.now().strftime(...)
```

5 모듈·표준 라이브러리 — 실습

Colab 워크시트 TODO

1 TODO 5-1 sensors.py
judge·parse_field 분리 + __main__ 자체 테스트

2 TODO 5-2 Counter 집계
모든 T 판정 → statuses → counts

3 TODO 5-3 로그 한 줄
datetime.strftime + f-string → log_line(t, status)

4 (확인) reload
고친 뒤 importlib.reload 로 반영

✅ **자동 채점** sensors.py 존재 · 함수/__main__ · counts · log_line (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 수정이 반영 안 되는 이유 → 캐시
- import 때 __main__ 블록 → 실행 안 됨

체크포인트 · 강사 확인

- 모듈 이름.함수 와 from import 를 구분해 쓰는가
- reload 를 스스로 떠올리는가
- '표준 라이브러리에 있지 않을까' 검색 습관

6 예외 처리

15:00–15:50

학습 목표

- try / except / else / finally 구조
- 구체적 예외로 잡기 · as e · 여러 except
- raise 와 사용자 정의 예외 (FrameError)
- 깨진 데이터는 세고 continue — 계속 진행

🎬 04 · 03 — 어제 멈춘 자리에서

- 04: 필드 누락 · 잘못된 숫자 — 어제는 여기서 멈췄다
- 03: [Day 2] 예외 전파와 try — parse → read → 호출자
- 프레임을 거슬러 올라가다 try 에서 잡히는 경로
- x 가 만들어지지 않는 것, '계속 진행' 출력 확인

핵심 개념

기본 구조

```
try:
    t = float("2S.3")
except ValueError as e:
    print("잡음:", e)      # 프로그램은 계속
```

사용자 예외

```
class FrameError(Exception): pass
if not (line.startswith("$") ...):
    raise FrameError(f"손상: {line!r}")
```

건너뛰고 계속

```
except (FrameError, ValueError):
    bad += 1; continue
```

원칙

```
except: 전부 잡기 금지 · 대처할 수 있는 곳에서 잡기
```

6 예외 처리 — 실습

Colab 워크시트 TODO

1 TODO 6-1 FrameError·parse_line
검증 실패 raise → (T, H, D) 반환

2 TODO 6-2 예외 구분
int("")·float("2S.3")·d[키]·x[10] → 맞는 except 로

3 TODO 6-3 건너뛰고 계속
Day 1 로그를 try 버전으로 — good 8 / bad 2

4 TODO 6-4 재시도 (심화)
flaky() 3회 재시도 → result 42, tries 3

✅ **자동 채점** parse_line/raise · errs 4종 · good/bad · result/tries (4항목)

🔍 자기 점검 (예측 → 애니메이션 → 코드)

- 예외를 안 잡으면 → 호출자로 전파
- finally → 항상 실행

체크포인트 · 강사 확인

- except 를 좁게 잡는 이유를 말하는가
- raise 를 '알리는 것'으로 이해하는가
- Day 1 과제 B 와 오늘 버전의 차이를 비교하게 한다

7 종합 실습 — 과제 A · B

16:00–16:50 · 2인 1조 허용, 각자 제출

과제 A · 센서 시뮬레이터 + 통계 (필수, 30분)

- make_reading(node_id) → {"id", "T", "H", "D"} 딕셔너리
- random.seed(42) 1회 후 노드 3개 × 10회 → readings
- stats_by_node → {id: {avg_T, max_T, alerts(T>35)}}
- report — f-string 표, 컴프리헨션 1회 이상, simulator.py 분리

```
node       avg_T   max_T  alerts
ESP32-01    26.4    33.1       0
ESP32-02    29.8    36.9       2
ESP32-03    25.1    31.0       0
```

✅ 4항목 — 학생 readings 기준 정합성 채점 (구조·사양·통계·alerts)

과제 B · 견고한 로그 리플레이 (선택, 15분)

- 필드 순서 뒤바뀐 줄 $H=60,T=25.3,D=120* + 빈 줄 혼입
- parse_line2 — 딕셔너리에 담아 순서 무관 파싱
- 빈 줄은 호출 전에 건너뜀 (예외 아님)
- err_counts = Counter() 로 예외 이름별 집계

```
ok 5개 · {'FrameError': 1, 'ValueError': 1}
```

✅ 4항목 · 막히면: 자료구조 D2-6 결정 트리 · 중첩 설계 D2-5 트리 뷰어

빨리 끝낸 조: 리포트 T 내림차순 정렬(4교시 lambda) · json.dump 저장

8 정리 · 퀴즈 · 제출

17:00–17:50

15분 — **과제 A 모범 답안 리뷰**
참조 실수 → 02 · 자료구조 선택 → D2-6 · seed 위치 오류

20분 — **Day 2 퀴즈 (Google Forms, 10문항)**
1번 참조 → 02 · 3번 .get → D2-1 · 7번 컴프리헨션 → D2-4 로 해설

5분 — **핵심 정리 · Day 3 예고**
dict 레코드가 내일 클래스가 됩니다

10분 — **제출**
Day2_이름.ipynb + 개념 워크시트 코드 7개 → '제출 요약' 셀 실행 후 저장

평가 기준 (Day 2 몫)

퀴즈 문항당 1점	10점
Colab TODO 자동 채점 1~6교시 24 + 과제 A 4 + 과제 B 4	32점
개념 워크시트 제출 오답 정리 캡처(노션) 또는 결과 코드	7×10
노트북 제출 미제출 시 실습 0점	필수

제출 폴더: 공유 Drive › 온디바이스AI_Python › Day2 › 제출
파일명 규칙: Day2_이름.ipynb
개념 워크시트:
…/anim/worksheet.html

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

리스트	b = a 는 이름표 · copy 는 겉만 · sort() 는 None, sorted() 는 새 리스트	02 · D2-3
딕셔너리	해시로 직행 · 없는 키는 .get() 방어 · 키는 불변만	D2-1
중첩	대괄호 하나 = 한 층 · 그 층의 자료형에 맞는 대괄호	D2-5
함수	호출 = 스택 프레임 · return 없으면 None · 기본값 [] 금지	03
컴프리헨션	for → if → 식 순서로 읽고, 원본은 그대로	D2-4
예외	구체적으로 잡고, 깨진 줄은 세고 continue · raise 는 알리기	03 · 04

Day 3 예고

클래스·파일·시스템 연동 — 9/4 (금)

- 오늘의 dict 레코드 {"id", "T", ...} 가 내일 클래스가 됩니다
- 파일 입출력 — 오늘 json.dump 를 본격적으로
- 시스템 연동 — 시리얼·프로세스, sensors.py 확장
- 예외 처리가 곳곳에서 다시 등장합니다

오늘 과제 (선택)

- 과제 B 미완이면 완성해 오기 (내일 출발 코드)
- simulator.py 결과를 json.dump 로 저장해 보기
- D2-6 결정 트리를 직접 답해 보며 복습

수고하셨습니다.